

Production Fullstack Guide

Django backend + Next.js / TypeScript frontend
What actually matters when you ship to real users.

Generated: May 2026

Contents

- Introduction
- Part A - Django in Production
- Part B - Next.js and TypeScript in Production
- Part C - Connecting Backend and Frontend
- Part D - The AI-Era Developer Mindset
- Part E - Production-Ready Checklist

Introduction

This guide is a focused reference for building production-grade systems with a Django backend and a Next.js / TypeScript frontend. It is not a tutorial and it is not about toy projects. Every item here is something that matters when real users, real data, and real money are involved.

The goal is simple: give you the high-leverage knowledge that separates code that "works on my machine" from code that survives production. In the age of AI coding agents you do not need to memorize every API, but you absolutely must understand these concepts so you can review, verify, and make the right architectural decisions. AI writes the code; you own the system.

How to read this guide: skim the section headers first, then go deep where you feel weak. The "Common pitfalls" lists at the end of each major part are the fastest way to find gaps in your current setup.

Part A - Django in Production

A1. Settings and configuration

Treat configuration as code that follows the 12-factor methodology: strict separation of config from code, and config lives in the environment.

- Split settings into base, development, and production modules, or use a single settings file driven entirely by environment variables. Pick one and be consistent.
- Read secrets from the environment using `django-environ` or `pydantic-settings`. Never hardcode secrets and never commit a real `.env` file. Commit a `.env.example` with dummy values instead.
- The non-negotiable production flags: `DEBUG` must be `False`, `SECRET_KEY` must come from the environment and be long and random, `ALLOWED_HOSTS` must list your real domains.
- Enforce HTTPS and secure cookies: `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS`, `SECURE_PROXY_SSL_HEADER` when behind a proxy.
- Fail loudly on missing required config at startup rather than failing mysteriously at request time.

```
# settings/production.py (env-driven)
import environ
env = environ.Env(DEBUG=(bool, False))

DEBUG = env("DEBUG")
SECRET_KEY = env("SECRET_KEY")
ALLOWED_HOSTS = env.list("ALLOWED_HOSTS")
DATABASES = {"default": env.db("DATABASE_URL")}

SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
```

A2. Project structure and the service layer

- Keep apps small and focused on a single domain (users, billing, orders). Avoid one giant app.
- Keep business logic out of views and out of serializers. Views handle HTTP, serializers handle shape

and validation, models handle persistence. Put real logic in a service layer (plain functions or service classes) so it can be tested and reused.

- Define a custom user model on day one, even if it is identical to the default. Changing it later is painful. Set `AUTH_USER_MODEL` before the first migration.
- Use selectors (read functions) and services (write functions) as a simple, testable boundary between your domain and the web layer.

A3. Database and the ORM

The database is where most production incidents and most performance problems live.

- Use PostgreSQL in production. Use the same engine locally; do not develop on SQLite and deploy on Postgres.
- Kill N+1 queries. Use `select_related` for foreign keys and `prefetch_related` for many-to-many and reverse relations. Profile with `django-debug-toolbar` or `django-silk`.
- Add indexes for every field you filter, order, or join on frequently. Use composite indexes that match your query patterns. Watch for unused indexes too.
- Use `only()` and `defer()` to avoid loading large columns you do not need. Use `values()` and `values_list()` when you do not need model instances.
- Use `bulk_create`, `bulk_update`, and `update()` to avoid per-row queries. Use `iterator()` for large result sets to control memory.
- Wrap multi-step writes in `transaction.atomic()`. Use `select_for_update()` to prevent race conditions on critical rows.
- Treat migrations as production artifacts: review them, never edit applied migrations, and be careful with data migrations and long-running schema changes that lock tables.

```
# Avoid N+1: one query instead of one-per-row
orders = (
    Order.objects
    .select_related("customer")
    .prefetch_related("items__product")
    .filter(status="paid")
)
```

A4. The API layer

- Choose Django REST Framework (mature, batteries included) or Django Ninja (fast, type hints, Pydantic). Both are solid; Ninja is lighter and more modern, DRF has the bigger ecosystem.
- Validate all input at the boundary. Never trust the client. Serializers (DRF) or Pydantic schemas (Ninja) are your validation gate.
- Always paginate list endpoints. Unbounded lists are a denial-of-service waiting to happen.
- Version your API from the start (URL path like `/api/v1/` is the simplest). Breaking changes go in a new version.
- Apply throttling and rate limits per user and per IP. Return proper status codes and a consistent error shape.
- Generate an OpenAPI schema with `drf-spectacular` (DRF) or the built-in schema (Ninja). This schema is also how you generate a typed frontend client.

A5. Authentication and security

- Choose your auth model deliberately. Session auth with `httpOnly` cookies is simplest and safest for a same-site web app. JWT (via `simplejwt`) suits mobile clients and cross-domain APIs but adds token rotation and revocation complexity.
- If you use JWT in a browser, store it in an `httpOnly`, `Secure`, `SameSite` cookie, never in `localStorage`,

to avoid XSS token theft.

- Keep CSRF protection on for any cookie-based auth. Configure CORS explicitly with `django-cors-headers`; never use a wildcard origin with credentials.
- Use Argon2 for password hashing (add `argon2-cffi` and put it first in `PASSWORD_HASHERS`). Enforce password validators.
- Enforce permissions at the object level, not just the view level. A user must not be able to read or edit another user's records by changing an ID.
- The ORM protects against SQL injection only if you avoid raw string formatting in `raw()` and `extra()`. Template autoescaping protects against XSS only if you do not mark untrusted data as safe.
- Add a Content Security Policy, set `X-Content-Type-Options`, and keep dependencies patched (`pip-audit`, `Dependabot`).

A6. Background tasks and caching

- Move slow or external work (emails, payments, image processing, third-party calls) out of the request cycle into Celery workers with Redis or RabbitMQ as the broker.
- Make tasks idempotent and safe to retry. Configure retries with backoff. Never assume a task runs exactly once.
- Use Celery Beat for scheduled jobs. Monitor the queue depth and worker health.
- Cache expensive reads in Redis. Cache at the right layer: per-view caching, low-level `cache.get/set` for computed values, or template fragment caching.
- The hard part of caching is invalidation. Prefer short TTLs and explicit invalidation on write over clever schemes. Always handle the cache-miss path correctly.

```
# tasks.py
from celery import shared_task

@shared_task(bind=True, max_retries=3, default_retry_delay=10)
def send_receipt(self, order_id):
    try:
        deliver_receipt(order_id)
    except TransientError as exc:
        raise self.retry(exc=exc)
```

A7. Performance and scaling

- Run with Gunicorn (WSGI) or Uvicorn/Gunicorn with Uvicorn workers (ASGI) if you use async views or websockets. Tune the worker and thread count to your CPU and workload; start with `workers = 2 x cores + 1`.
- Set sane timeouts. A slow upstream call should not pin a worker forever.
- Serve static files with WhiteNoise or a CDN, and serve user media from object storage (S3 or compatible), never from the container's local disk.
- Use `CONN_MAX_AGE` for persistent DB connections, and put PgBouncer in front of Postgres for connection pooling at scale.
- Scale reads with read replicas and a database router once a single primary is no longer enough.
- Profile before optimizing. Measure with `silk` or `APM`; do not guess.

A8. Testing and code quality

- Use `pytest` with `pytest-django`. Use `factory_boy` for test data instead of fixtures you maintain by hand.
- Test the service layer and the API contract. Aim for meaningful coverage of business rules, not a coverage percentage for its own sake.
- Enforce style and types automatically: `ruff` (lint + format) or `black` + `ruff`, plus `mypy` for type checking.

Wire it all into pre-commit so bad code never reaches the repo.

- Keep tests fast and isolated. Use a real Postgres in CI, not SQLite, so tests match production behavior.

A9. Observability

- Log in structured JSON with a request ID on every line so you can trace a single request across services. Never log secrets, tokens, or full card numbers.
- Capture and triage errors with Sentry. Set up alerts that page a human only for real problems.
- Expose a health check endpoint for liveness and readiness probes. Export metrics (Prometheus) for request rate, latency, error rate, and queue depth.
- Track the four golden signals: latency, traffic, errors, and saturation.

A10. Deployment and infrastructure

- Containerize with a multi-stage Dockerfile: build dependencies in one stage, copy only what you need into a slim runtime image. Run as a non-root user.
- On deploy, run collectstatic and run migrate exactly once (not once per replica). Use a release/migration step or a job, not the app startup, to run migrations.
- Put Nginx or a load balancer in front of Gunicorn. Terminate TLS at the edge. Enable gzip and proper cache headers.
- Aim for zero-downtime deploys (rolling or blue-green). Make migrations backward compatible so old and new code can run together briefly.
- Manage secrets with a real secrets manager (AWS Secrets Manager, Vault, or your platform's equivalent), injected as environment variables at runtime.

A11. Django common pitfalls

- `DEBUG=True` in production (leaks stack traces and settings).
- Empty or wildcard `ALLOWED_HOSTS`, or committing `SECRET_KEY`.
- N+1 queries and missing database indexes.
- Blocking calls (HTTP, email, heavy compute) inside the request cycle instead of a task queue.
- Storing uploaded media on the container filesystem, which disappears on redeploy.
- Running migrations concurrently from multiple replicas.
- Object-level permission gaps that let users access other users' data by ID.

Part B - Next.js and TypeScript in Production

B1. Rendering model (App Router)

- Default to Server Components. They run on the server, keep secrets and heavy logic off the client, and ship zero JavaScript for static parts. Add "use client" only when you need interactivity, browser APIs, state, or effects.
- Understand the rendering strategies: static rendering (build time), dynamic rendering (per request), and incremental static regeneration (ISR) with `revalidate`. Choose per route based on how fresh the data must be.
- Use streaming and `Suspense` to send the shell quickly and stream slower data in, improving perceived performance.
- Keep client components small and at the leaves of your tree. A "use client" at the top of a tree turns everything below it into client code.

B2. Data fetching and caching

- Fetch on the server in Server Components using `fetch` with explicit caching: cache for static data,

revalidate: N for time-based, and no-store for always-fresh data. Know which one each request needs.

- Use Route Handlers for your own API endpoints and Server Actions for mutations triggered from forms and components.
- Avoid request waterfalls. Fetch in parallel with Promise.all when requests are independent.
- For client-side server state (polling, infinite scroll, optimistic updates), use TanStack Query (React Query) or SWR. Do not hand-roll fetch-in-useEffect for anything non-trivial.
- Revalidate caches after mutations with revalidatePath or revalidateTag so the UI reflects new data.

```
// Server Component: parallel fetch, time-based revalidation
async function Dashboard() {
  const [stats, feed] = await Promise.all([
    fetch(api("/stats"), { next: { revalidate: 60 } }).then(r => r.json()),
    fetch(api("/feed"), { cache: "no-store" }).then(r => r.json()),
  ]);
  return <DashboardView stats={stats} feed={feed} />;
}
```

B3. TypeScript discipline

- Turn on strict mode and keep it on. The whole point of TypeScript is lost without it.
- Ban any. When you do not know a type, use unknown and narrow it. Treat eslint no-explicit-any as an error.
- Types are erased at runtime, so validate all external data (API responses, form input, URL params, env vars) at the boundary with Zod. A type assertion is a promise you cannot keep; a Zod parse is a check you can trust.
- Type your environment variables with a schema (for example t3-env) so a missing variable fails the build, not production.
- Share types with the backend by generating them from the OpenAPI schema (openapi-typescript). One source of truth, no drift.

B4. State and forms

- Separate server state from client state. Server state (data from your API) belongs in React Query/SWR. Client state (UI toggles, wizard steps) belongs in component state or a small store like Zustand or Jotai. Do not put server data in a global store and try to keep it in sync by hand.
- Build forms with react-hook-form for performance and Zod for validation, sharing the same schema between client and server.
- Implement optimistic updates carefully and always handle the rollback path on error.

B5. Styling, UI, and accessibility

- Tailwind CSS is the de facto styling approach. Pair it with a component layer like shadcn/ui (built on Radix) for accessible, unstyled primitives you own and customize.
- Accessibility is a production requirement, not a nice-to-have: semantic HTML, labels on inputs, keyboard navigation, focus management, and sufficient color contrast. Many a11y wins are also SEO wins.
- Keep a consistent design system: spacing scale, color tokens, and typography defined once.

B6. Performance and Core Web Vitals

- Optimize the three Core Web Vitals: LCP (largest contentful paint), CLS (cumulative layout shift), and INP (interaction to next paint).
- Use next/image for automatic resizing, lazy loading, and modern formats. Always set width and

height to avoid layout shift.

- Use next/font to self-host fonts and eliminate render-blocking font requests and layout shift.
- Reduce client JavaScript: prefer Server Components, code-split with next/dynamic, and lazy-load heavy widgets (maps, editors, charts).
- Analyze your bundle (@next/bundle-analyzer). Watch for large dependencies pulled into client components.
- Memoize only where profiling shows a need. Premature memoization adds complexity without benefit.

B7. Authentication on the frontend

- Use Auth.js (NextAuth) for common providers, or integrate your Django auth via httpOnly cookies. Store session tokens in httpOnly, Secure cookies, never in localStorage.
- Protect routes in middleware for a fast, centralized redirect, and re-check authorization on the server for every sensitive action. Never trust the client to enforce access.
- Send CSRF protection on mutations when using cookie auth. Keep the auth flow consistent between server and client components.

B8. SEO and metadata

- Use the Metadata API (static metadata export or generateMetadata) for titles, descriptions, and Open Graph tags.
- Generate sitemap.xml and robots.txt. Add structured data (JSON-LD) for rich results where relevant.
- Use dynamic Open Graph images for shareable links. Set canonical URLs to avoid duplicate-content issues.
- For multi-language sites, use the i18n routing patterns and set hreflang correctly.

B9. Project structure and quality

- Organize by feature, not by file type, once the app grows. Co-locate components, hooks, and tests with the feature they belong to.
- Enforce ESLint and Prettier, set up absolute imports, and add Husky with lint-staged so commits are clean.
- Understand env var exposure: only variables prefixed with NEXT_PUBLIC_ are sent to the browser. Everything else stays server-only. Never put a secret behind NEXT_PUBLIC_.

B10. Error handling and UX states

- Use the special files: loading.tsx for suspense fallbacks, error.tsx for route error boundaries, and not-found.tsx for 404s. Always handle the loading and error states; a spinner that never resolves is a bug.
- Show meaningful empty states and error messages. Use toasts for transient feedback.
- Add a global error boundary so one broken component does not blank the whole page.

B11. Testing the frontend

- Unit and component tests with Vitest (or Jest) plus React Testing Library. Test behavior the user sees, not implementation details.
- End-to-end tests with Playwright for critical flows (sign up, checkout, core navigation).
- Mock network with MSW so tests are deterministic and do not depend on a live backend.

B12. Deployment

- Vercel is the lowest-friction host and matches Next.js features one-to-one. Self-hosting is fully supported via the standalone output and a Node server in a Docker container, or a static export for fully static sites.

- Know your runtimes: the Edge runtime is fast and global but has a limited API surface; the Node runtime is full-featured. Choose per route.
- Set caching and CDN headers deliberately. Understand the cost and behavior of image optimization at scale.
- Monitor real users with Sentry and a web-vitals/analytics tool. Watch bundle size and Core Web Vitals over time, not just at launch.

B13. Next.js common pitfalls

- Leaking secrets by prefixing them with `NEXT_PUBLIC_`.
- Marking everything "use client" and shipping a huge bundle, defeating the point of Server Components.
- Fetching in `useEffect` when a Server Component fetch would be simpler, faster, and SEO-friendly.
- No explicit caching strategy, leading to either stale data or no caching at all.
- Hydration mismatches from rendering different markup on server and client (for example using `Date.now()` or `window` during render).
- Ignoring loading and error states, and shipping layout shift from unsized images.

Part C - Connecting Backend and Frontend

C1. The API contract

- The OpenAPI schema generated by Django is the contract. Generate the TypeScript client and types from it (`openapi-typescript` or `orval`) so the frontend and backend never drift. When the API changes, the frontend fails to compile, which is exactly what you want.
- Define a single, consistent error response shape on the backend and handle it in one place on the frontend.

C2. Auth across the stack

- For a same-site web app, cookie-based sessions are the simplest secure choice: Django sets an `httpOnly` cookie, the browser sends it automatically, and CSRF protection covers mutations.
- For cross-domain or mobile, use JWT with short-lived access tokens and refresh tokens, and a clear rotation and revocation strategy.
- Configure CORS and CSRF together and test them early; they are the most common source of "works locally, breaks in prod" auth bugs.

C3. Environments and delivery

- Maintain clear environments: local, staging, production, each with its own config and database. Never test against production data.
- Consider a monorepo (Turborepo) to share types and tooling between frontend and backend, or keep separate repos with a published types package. Either works; pick based on team size.
- Use Docker Compose for a one-command local stack (Postgres, Redis, backend, frontend). Match local versions to production.
- Build one CI/CD pipeline that lints, type-checks, tests, builds, and deploys. Block merges on red.

Part D - The AI-Era Developer Mindset

D1. What changes and what does not

AI agents change how fast you produce code. They do not change who is responsible for it. You can

generate a feature in minutes, but you still own its security, performance, correctness, and fit within the system. The bottleneck moves from typing to judgment.

What AI is great at: scaffolding, boilerplate, writing tests from a spec, explaining unfamiliar code, translating between frameworks, and proposing options. Lean on it for these.

What stays your job: system design, data modeling, security decisions, choosing trade-offs, defining the contract, and verifying that the generated code is actually correct for your context.

D2. Always review AI-generated code for

- Security: is auth enforced at the object level, is input validated, are there injection or XSS holes, are secrets handled correctly.
- Data and performance: does it introduce N+1 queries, missing indexes, unbounded lists, or blocking calls in the request path.
- Correctness and edge cases: empty inputs, concurrency, error paths, and failure modes the happy-path demo never hits.
- Fit: does it match the project's patterns, or did it invent a new way of doing something you already solved.

D3. Fundamentals AI does not replace

- How HTTP works: methods, status codes, headers, caching, cookies.
- Databases and SQL: indexes, transactions, the query the ORM actually runs.
- Security basics: the OWASP top risks and how your stack mitigates each.
- Debugging: reading a stack trace, bisecting a problem, forming and testing a hypothesis.
- System design: where state lives, how services talk, what fails and how.

D4. A healthy workflow with AI

- Work in small, reviewable diffs. A 2000-line AI dump is unreviewable and therefore unsafe.
- Keep a strong automated safety net: types, linters, and tests catch a large share of AI mistakes before you do.
- Write or generate tests for the behavior, then let AI iterate against them.
- Read the docs for the critical 20 percent (auth, payments, data model). Let AI handle the routine 80 percent, with your review.

Part E - Production-Ready Checklist ---

E1. Backend checklist

- DEBUG is False, SECRET_KEY and all secrets come from the environment, ALLOWED_HOSTS is set.
- HTTPS enforced, secure and httpOnly cookies, HSTS enabled.
- Custom user model, object-level permissions, throttling and rate limits.
- PostgreSQL, indexes for hot queries, no N+1, transactions on multi-step writes.
- Background tasks for slow work, idempotent and retryable.
- Structured logging with request IDs, Sentry, health checks, metrics.
- Multi-stage Docker image, non-root user, migrations run once per release, static and media off local disk.
- CI runs lint, type-check, and tests against a production-like database.

E2. Frontend checklist

- TypeScript strict mode, no any, runtime validation at boundaries with Zod.
- Server Components by default, minimal and leaf-level client components.
- Explicit caching and revalidation strategy per route, no fetch-in-useEffect for server state.
- next/image and next/font in use, bundle analyzed, Core Web Vitals tracked.
- Auth tokens in httpOnly cookies, route protection in middleware plus server-side checks.
- Metadata, sitemap, robots, and structured data configured.
- loading, error, and not-found states handled everywhere.
- Vitest/RTL plus Playwright e2e for critical flows, MSW for mocking.

E3. A pragmatic default stack

- Backend: Django + Django REST Framework or Django Ninja, PostgreSQL, Redis, Celery, Gunicorn/Uvicorn, Docker, Sentry.
- Frontend: Next.js (App Router) + TypeScript strict, Tailwind + shadcn/ui, TanStack Query, react-hook-form + Zod, Auth.js or cookie sessions.
- Glue: OpenAPI to typed client, Docker Compose locally, one CI/CD pipeline, staging before production.

Build it small, ship it safe, measure it in production, and let AI accelerate the parts you already understand.